1. Software Engineering Management for Increased E-Commerce Security and to Overcome E-Commerce Threats

Authors, year, where published: Mohammad Akbar, 2020, Cross Current International Journal of Economics, Management and Media Studies, Vol. 2 Issue 2, pp. 42–44

Type of study: Review (short, 3 pages)

Who they studied: Nobody. It summarises earlier papers on e-commerce security and trust.

Main finding: Privacy, integrity, confidentiality and non-repudiation are the main security dimensions for e-commerce. Firewalls, VPNs and anti-virus are the usual protections, but the author says no single measure is enough. Encryption, digital signatures, verification and intrusion detection are also needed.

Engineer vs manager: Says almost nothing about either role. Despite "software engineering management" in the title, the paper never discusses how managers or engineers handle security.

Useful for us because: Only as background that security and customer trust are major barriers. It has no evidence of its own.

Quote or number worth citing: A Trojan horse lets an attacker see cleartext before it is encrypted, which defeats password protection and encryption (p. 43). No numbers.

Warning: Weak source. The header says Issue 2 but the page footer says 2(1). The text cites "Godwin (2001)" but the reference list has Udo (2001).

2. Software Testing and Release Decision at Different Statistical Confidence Levels with Consideration of Debuggers' Learning and Negligent Factors

Authors, year, where published: Chun-Wu Yeh and Chih-Chiang Fang, 2024, International Journal of Industrial Engineering Computations, Vol. 15, pp. 105–126

Type of study: Mathematical modelling (a software reliability growth model), tested on 8 published datasets plus a made-up example company

Who they studied: No people. They used 8 historical failure datasets from earlier studies (e.g. a telecom system, Mozilla Firefox 3.5 testing data, an open-source project management tool).

Main finding: Their model adds a learning factor (testers get better at spotting known error patterns) and a negligent factor (debuggers introduce new errors while fixing). It fits both S-shaped and exponential data and ranked in the top two of four models on most datasets. They also propose confidence intervals that narrow over time, so managers can plan for a worst case.

Engineer vs manager: Both, indirectly. The learning and negligence factors describe engineers' (testers'/debuggers') behaviour. The release-time decision model is a tool for managers.

Useful for us because: It links engineer skill and training directly to manager decisions on cost and release timing. The authors say improving learning and detection needs more training or senior staff, which costs money, so managers face a trade-off.

Quote or number worth citing: In the example, the best plan (P2) gives an optimal release at 3.45 months for $125,441, reaching 95.38% reliability after the warranty. A cautious worst-case view pushes release to 3.7 months (p. 121).

3. Factors Affecting the Adoption of Cloud for Software Development: A Case from Turkey

Authors, year, where published: Erhan Pisirir, Oumout Chouseinoglou, Cüneyt Sevgi and Erkan Uçar, 2023, International Journal of Information Technology & Decision Making (World Scientific), DOI 10.1142/S0219622023500517

Type of study: Survey (personally administered questionnaire), analysed with Structural Equation Modeling (SEM)

Who they studied: 191 developers, project managers and senior managers from 30 software organisations in Turkey, covering 84 projects. This gave 268 valid responses after cleaning. 83.2% of respondents were male.

Main finding: The model combines TAM, TOE and the authors' new Personal–Organization–Project (POP) factors, and explains 69.6% of intention to use the cloud. 15 of 21 hypotheses were accepted. Perceived suitability is the strongest driver of intention, and top management support is the strongest driver of actual use. Actual cloud use in Turkey is still low.

Engineer vs manager: Directly compares both. Developers find the cloud more suitable for larger projects and projects with more licensed software, but management is less likely to migrate. The authors suggest a sunk-cost effect: managers who bought expensive licences resist switching.

Useful for us because: It's the strongest source here. It has a real sample, includes both roles, and shows that what developers want depends on what managers support.

Quote or number worth citing: Top management support → actual use, standardised weight 0.636 (p. 2350051-21). Perceived suitability → intention, 0.644 (p. 2350051-18).

4. Project Risk Management in Iranian Software Projects

Authors, year, where published: Ali Reza Afshari, Vladimir Brtka and Melita Ćoćkalo-Hronjec, 2018, Journal of Engineering Management and Competitiveness (JEMC), Vol. 8 No. 2, pp. 81–88

Type of study: Survey (structured questionnaire, done as interviews)

Who they studied: 30 software project managers or IT managers in Iran (a planned sample of 50 dropped to 30). When a manager was too busy, a senior developer answered instead.

Main finding: Risk assessment is poorly practised. Most managers deal with risks only as they appear, don't document them, and don't reuse past experience. The authors propose a simple risk assessment framework and tool.

Engineer vs manager: Mostly the manager view. But many of the risks managers listed are about engineers: over-reliance on a few skilled developers, high staff turnover and staff lacking the right skills.

Useful for us because: It's a manager-only source, like Paper 6 in Sirjana's notes, and shows managers blaming user and staffing issues for delays.

Quote or number worth citing: 83% of managers assess risk only implicitly and use an ad hoc approach; 93% have never used a risk tool (pp. 86–87).

Warning: Small, purposive sample from one country, so results are weak evidence.

5. RAG-Based AI Agents for Enterprise Software Development: Implementation Patterns and Production Deployment

Authors, year, where published: Xiuyuan Zhao, Tiejiang Sun, Shaochen Ren, Jingyun Yang and Yang Liu, 2025, Frontiers in Artificial Intelligence Research, Vol. 2 Issue 3, pp. 501–520

Type of study: Review

Who they studied: Nobody. It's a technical overview of AI agents that use retrieval-augmented generation (RAG).

Main finding: RAG agents can help with code completion, test generation, documentation and code review. Deploying them in companies brings problems with speed, cost, security, hallucinations and evaluation.

Engineer vs manager: Mostly about developers as users. The "organizational change management" section touches on managers: training, pilot programs, team champions and developers' fears about job loss.

Useful for us because: Background on how AI may change developers' work, similar to Paper 5 in Sirjana's notes. No evidence about real engineers or managers.

Quote or number worth citing: Nothing reliable.

Warning: Weak source. Figure captions read like unfinished instructions ("Diagram should illustrate..."), the reference list has a stray sentence, and some references are off-topic (tall buildings, tax fraud). Candidate for replacement.

Conclusions

- **Engineers and managers together:** Paper 3 only. Paper 2 models engineer behaviour inside a manager's decision tool.
- **Managers only:** Paper 4.
- **Nobody studied:** Papers 1 and 5 (reviews).
- **Strongest sources:** Paper 3, then Paper 2 and Paper 4.
- **Weak sources:** Papers 1 and 5 are low-tier and candidates for "no" and a replacement. Paper 4 could be kept as weak evidence.
- **Key references to look up:** Schmidt, Lyytinen, Keil and Cule (2001), an international Delphi study of software project risks (from Paper 4); Tiwana and Keil (2004), the one-minute risk assessment tool (Paper 4); Butt et al. (2019) on cloud for agile development (Paper 3); Chiu, Huang and Lee (2008) on learning effects in testing (Paper 2).

Interview ideas

- Paper 3 says management support matters most for actually using new tools. Do engineers feel blocked by managers, and do managers see it that way?
- Paper 3 suggests managers stick with paid tools because of sunk cost. Do professionals recognise this?
- Paper 4 says managers handle risks only when they appear. Do engineers see risks earlier than managers do?
- Paper 2 says training improves testers but costs money. Who should decide how much to invest: engineers or managers?
- Paper 5 mentions developers fearing job loss to AI. Do managers and engineers see AI tools differently?